ALGORITMA GENETIKA (GA)

Disusun untuk memenuhi Tugas Laporan Mata Kuliah Kecerdasan Buatan

Kelompok 14:

Izzan Maula Rifqi 103122430009

Muhammad Akbar Ivanka 103122400069

PROGRAM STUDI S1 SOFTWARE ENGINEERING

TELKOM UNIVERSITY PURWOKERTO

A. Deskripsi

Algoritma heuristik berbasis populasi seperti Algoritma Genetika sering digunakan untuk menyelesaikan permasalahan optimasi yang kompleks dan multimodal. Studi kasus ini berfokus pada pencarian nilai minimum dari fungsi matematika non-linear dua variabel berikut:

Fungsi yang dievaluasi didefinisikan sebagai berikut:

$$f(x1, x2) = -\left(sin(x1)cos(x2)\tan(x1 + x2) + \frac{1}{2}exp\left(1 - \sqrt{x}2^2\right)\right)$$

Adapun ruang pencarian dibatasi oleh domain (batas nilai) berikut:

- $-10 \leq x1 \leq 10$
- $-10 \leq x2 \leq 10$

Fungsi ini mengandung kombinasi fungsi trigonometri (sin, cos, tan) dan fungsi eksponensial, sehingga memiliki banyak local minima yang dapat menyebabkan metode pencarian berbasis gradien mudah terjebak. Oleh karena itu, Algoritma Genetika dipilih karena kemampuannya melakukan pencarian global secara paralel melalui mekanisme evolusi (seleksi, crossover, dan mutasi) tanpa bergantung pada informasi turunan fungsi.

Tujuan utama program adalah mengimplementasikan Algoritma Genetika dari nol (tanpa library khusus seperti DEAP atau PyGAD) menggunakan modul standar Python (random dan math) untuk menemukan pasangan (x1,x2) yang menghasilkan nilai f(x1 ,x2) sekecil mungkin.

B. Analisis dan Desain Algoritma Genetika

Untuk merancang algoritma yang dapat beroperasi secara optimal, penetapan arsitektur GA disusun melalui justifikasi desain berikut :

2.1 Representasi Kromosom dan Dekode

- Desain Kromosom merupakan Individu yang direpresentasikan menggunakan pengkodean biner dengan panjang total 26 bit.
- Pembagian Gen pada Kromosom dibelah secara simetris; 13 bit pertama dialokasikan untuk memetakan nilai x1, dan 13 bit kedua untuk nilai x2.
- Analisis Presisi Penggunaan 13 bit per variabel menghasilkan $2^{13} - 1 = 8.191$ kemungkinan titik diskrit. Mengingat rentang domain adalah 20 (dari -10 hingga 10), maka tingkat presisi yang didapat adalah $\frac{20}{8.191} = 0.0024$ Resolusi ini sangat

presisi untuk. pencarian nilai desimal koma tanpa membebani memori komputasi secara berlebihan.

- Proses Dekode: Nilai biner diubah menjadi desimal menggunakan Least Significant Bit (LSB), lalu dipetakan ke dalam domain menggunakan rumus pemetaan linier:

$$x = X_{min} + \left(\frac{X_{max} - X_{min}}{2^{panjanggen} - 1}\right) \times NilaiDesimal$$

2.2 Ukuran Populasi dan Kriteria Penghentian

- Ukuran Populasi awal yang cukup masif, yaitu 125 individu (berbeda dengan standar umum 50). Fungsi trigonometri pada soal memiliki fluktuasi yang sangat tajam. Populasi yang besar di awal menjamin terciptanya keanekaragaman genetik (genetic diversity) yang luas untuk memetakan seluruh area kurva.
- Kriteria Penghentian (50 Generasi) Batas waktu evolusi ditetapkan pada iterasi ke-50. Dengan 125 individu dikalikan 50 generasi, sistem akan melakukan 6.250 kali evaluasi fungsi. Kombinasi ini sangat efektif, populasi besar memastikan eksplorasi ruang yang luas, sementara generasi yang lebih pendek mencegah komputasi membuang waktu saat populasi sudah mulai konvergen.

2.3 Transformasi Fungsi Fitness

Secara arsitektur, Algoritma Genetika dirancang untuk mencari individu dengan nilai fitness maksimum (Maximization). Karena permasalahan studi kasus menuntut nilai terkecil (Minimization), maka perlu melakukan transformasi inversi pada nilai fungsi objektif:

$$Fitness = -f(x_1, x_2)$$

Melalui negasi ini, kromosom yang menghasilkan nilai fungsi objektif sangat kecil (negatif dalam) akan memiliki nilai fitness yang sangat besar secara matematis, sehingga sistem GA akan memprioritaskan individu tersebut.

2.4 Mekanisme Seleksi Induk (Parent Selection)

Metode yang digunakan adalah Tournament Selection dengan parameter jumlah peserta k = 3. Tiga kromosom dipilih secara acak, dan kromosom dengan fitness tertinggi mutlak akan menjadi induk.

Pada fungsi dengan nilai negatif atau rentang yang rapat, Roulette sering gagal menyeleksi dengan adil. Tournament Selection memecahkan masalah ini karena seleksi murni didasarkan pada peringkat relatif (siapa yang lebih besar), bukan proporsi skalanya.

2.5 Probabilitas Operasi Genetik (Pc dan Pm)

Parameter Operasi	Nilai	Argumen dan Analisis Pemilihan
Probabilitas Crossover (Pc)	0.85	Menggunakan metode Single-point Crossover. Nilai 85% memastikan tingkat eksploitasi yang sangat agresif. Induk-induk yang unggul akan hampir selalu bertukar informasi genetik (titik potong acak antara gen 1 hingga 25) untuk melahirkan kombinasi anak yang lebih sempurna.
Probabilitas Mutasi (Pm)	0.06	Menggunakan metode Bit-flip Mutation. Peluang 6% per bit pada kromosom 26-bit berarti secara matematis akan ada 1.56 bit yang terbalik (mutasi) pada setiap individu baru. Ini adalah angka ideal untuk menyuntikkan keberagaman guna mencegah local minima, tanpa merusak kromosom secara brutal.

2.6 Strategi Pergantian Generasi

Sistem menggunakan metode Generational Replacement berbasis Elitism. Setiap kali generasi baru dibentuk, seluruh populasi lama akan dihapus dan diganti oleh keturunan baru. Namun, 2 individu dengan nilai fitness paling tinggi dari generasi sebelumnya akan langsung masuk ke populasi baru. Hal ini memberikan garansi matematis bahwa skor terbaik yang pernah dicapai sistem tidak akan pernah menurun akibat probabilitas acak dari crossover maupun mutasi.

C. Analisis dan Desain Algoritma Genetika

Program dibangun murni dari awal menggunakan Python, mengandalkan library bawaan random dan math. Berikut adalah pemetaan fungsional kodenya :

1. `bikin_populasi_awal()`: Membangkitkan array acak berisi 125 list, di mana setiap list terdiri dari 26 integer (0 atau 1).
2. `dekode_kromosom()`: Membelah array biner menjadi gen x1 dan x2, lalu melakukan perhitungan linear mapping untuk mengembalikan nilai desimal aktual.
3. `function_objektif()` & `total_fitness()` : Menerima input desimal, memasukkannya ke rumus sin, cos, tan, dan exp, serta menangkap potensi error (seperti pembagian tak hingga) menggunakan blok `try-except`.
4. Siklus Evolusi (`run_algoritma_genetik()`) : Sebuah iteration loop (`for`) yang di dalamnya mengeksekusi pengurutan fitness (Elitism), pemanggilan fungsi `seleksi_kromosom`, pertukaran gen lewat `crossover()`, pengacakan bit lewat `mutasi()`, hingga populasi baru genap 125 individu dan menggantikan populasi lama.

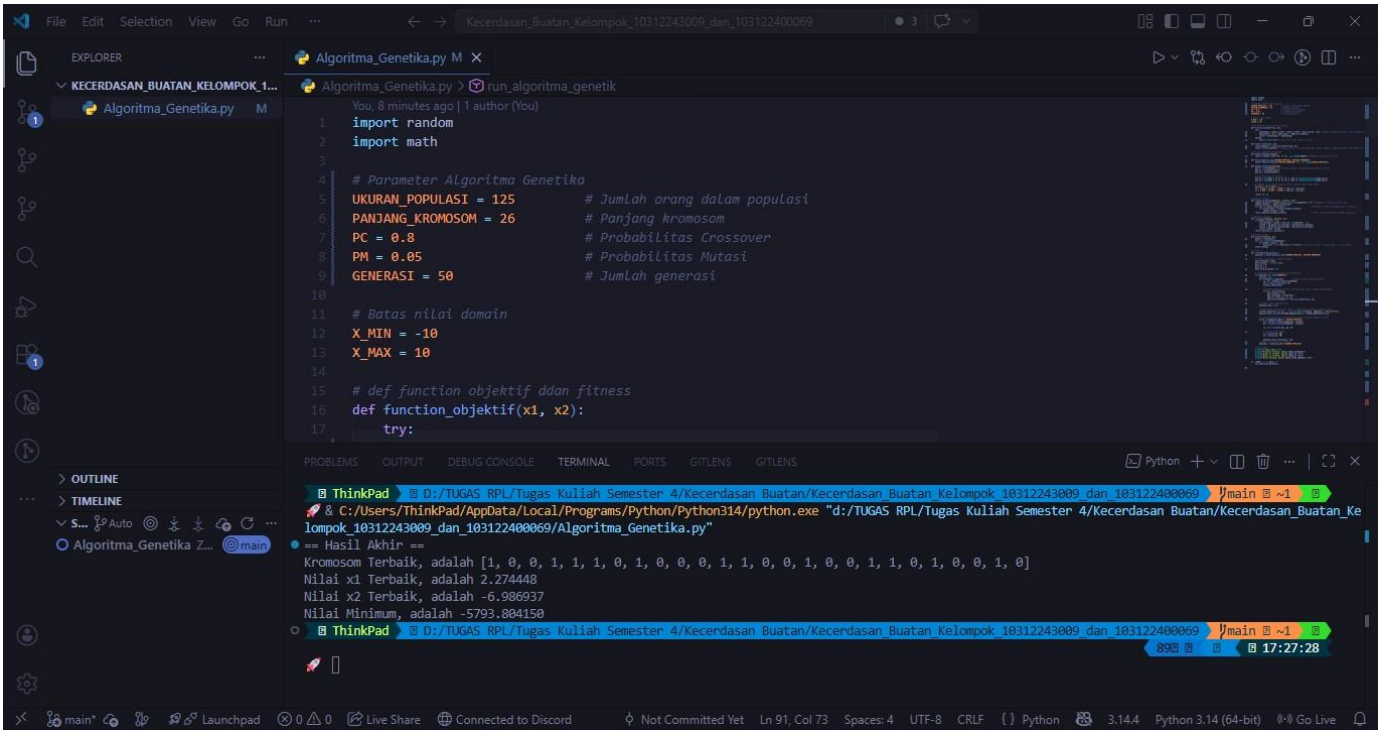
D. Hasil dan Output Program

Program dijalankan dengan parameter sebagai berikut:

Parameter	Nilai	Keterangan
Ukuran Populasi	125	Jumlah individu per generasi
Panjang Kromosom	26 bit	13 bit untuk x1 , 13 bit untuk x2
Jumlah Generasi	50	Kriteria penghentian
Probabilitas Crossover (Pc)	0.8	Single-point crossover
Probabilitas Mutasi (Pm)	0.05	Bit-flip per bit
Domain	[-10,10]	Batas nilai x1dan x2

Berdasarkan eksekusi program dengan parameter yang sudah ditentukan. Algoritma Genetika berhasil mengeksplorasi ruang pencarian dan menemukan kombinasi nilai secara optimal.

Meskipun GA bersifat probabilistik (hasil run bisa sedikit berbeda akibat seed acak), sistem secara konsisten menunjukkan konvergensi ke arah nilai minimum f(x).



E. Pembagian Peran Anggota Kelompok

Nama Anggota	NIM	Peran
Izzan Maula Rifqi	103122430009	Menganalisis parameter Algoritma genetika, penulisan laporan serta implementasi Python
Muhammad Akbar Ivanka	103122400069	implementasi Python, pengujian algoritma program, serta penulisan laporan